

1. Core Session & Context Modules

Required for initializing and managing the Spark application lifecycle.

- **SparkSession** (pyspark.sql)
 - **Functionality:** Creates, configures, and manages a PySpark application session.
 - **Syntax:**
`SparkSession.builder.appName("name").master("mode").getOrCreate()`
 - **Example:** `spark = SparkSession.builder.appName("test").master("local[*]").getOrCreate()`
- **spark.read** (pyspark.sql.SparkSession)
 - **Functionality:** Creates a DataFrameReader object used to load external data sources into DataFrames.
 - **Syntax:** `spark.read.option("key", "value").csv("path") / .json("path")`
 - **Example:** `df = spark.read.option("header", "true").csv("data.csv")`
- **sparkContext.parallelize()** (pyspark.context.SparkContext)
 - **Functionality:** Converts a native Python collection (like a list) into a distributed Resilient Distributed Dataset (RDD).
 - **Syntax:** `spark.sparkContext.parallelize(data_list)`
 - **Example:** `rdd = spark.sparkContext.parallelize([1, 2, 3])`
- **createDataFrame()** (pyspark.sql.SparkSession)
 - **Functionality:** Converts an RDD or a standard Python list into a PySpark DataFrame with a structured schema.
 - **Syntax:** `spark.createDataFrame(rdd_or_list, schema)`
 - **Example:** `df = spark.createDataFrame(rdd, ["id", "name"])`
- **spark.sql()** (pyspark.sql.SparkSession)
 - **Functionality:** Executes an ANSI SQL query directly against registered temporary views and returns the result as a DataFrame.
 - **Syntax:** `spark.sql("SELECT_QUERY")`
 - **Example:** `df = spark.sql("SELECT customer FROM orders")`
- **spark.stop()** (pyspark.sql.SparkSession)

- **Functionality:** Terminates the running Spark session and releases all associated cluster resources.
 - **Syntax:** `spark.stop()`
 - **Example:** `spark.stop()`
-

⚙️ 2. Structural & Column Manipulation

Used to select, update, type-cast, clean, and modify structural columns.

- **col()** (`pyspark.sql.functions`)
 - **Functionality:** Selects and references a specific DataFrame column by its name for programmatic transformations.
 - **Syntax:** `col("column_name")`
 - **Example:** `df.select(col("amount"))`
- **lit()** (`pyspark.sql.functions`)
 - **Functionality:** Creates a literal or static constant value column inside a PySpark DataFrame.
 - **Syntax:** `lit(value)`
 - **Example:** `df.withColumn("constant_val", lit(0))`
- **withColumn()** (`pyspark.sql.DataFrame`)
 - **Functionality:** Adds a new column or updates an existing column of matching name based on a functional expression.
 - **Syntax:** `df.withColumn("new_col_name", expression)`
 - **Example:** `df.withColumn("price_double", col("price").cast("double"))`
- **cast()** (`pyspark.sql.Column`)
 - **Functionality:** Changes the data type of an existing column to a target primitive or structural data type.
 - **Syntax:** `col("col_name").cast("target_type")`
 - **Example:** `df.withColumn("id_str", col("id").cast("string"))`
- **coalesce()** (`pyspark.sql.functions`)
 - **Functionality:** Evaluates arguments sequentially and returns the first non-null column value encountered.

- **Syntax:** `coalesce(col_1, col_2, default_lit)`
 - **Example:** `df.withColumn("clean_bonus", coalesce(col("bonus"), lit(0.0)))`
 - **trim()** (`pyspark.sql.functions`)
 - **Functionality:** Trims leading and trailing whitespaces from strings.
 - **Syntax:** `trim(col("col_name"))`
 - **Example:** `df.withColumn("clean_city", trim(col("city")))`
 - **when().otherwise()** (`pyspark.sql.functions`)
 - **Functionality:** Evaluates sequential conditional expressions, acting like an if-else statement.
 - **Syntax:** `when(condition, value).otherwise(default_value)`
 - **Example:** `df.withColumn("status", when(col("age") >= 18, "Adult").otherwise("Minor"))`
 - **udf()** (`pyspark.sql.functions`)
 - **Functionality:** Registers a native custom Python function to execute over DataFrame rows (best avoided for performance unless necessary).
 - **Syntax:** `udf(python_function, target_return_datatype)`
 - **Example:** `clean_udf = udf(lambda x: x.strip().title(), StringType())`
-

3. Row Actions, Filters, & Aggregations

Used to sort, slice, dice, pivot, and execute group metrics over rows.

- **filter()** / **where()** (`pyspark.sql.DataFrame`)
 - **Functionality:** Filters rows based on a boolean condition (both work identically).
 - **Syntax:** `df.filter(condition)`
 - **Example:** `df.filter(col("status") != "ERROR")`
- **drop_duplicates()** / **distinct()** (`pyspark.sql.DataFrame`)
 - **Functionality:** Returns a new DataFrame containing unique rows by removing completely identical duplicates.
 - **Syntax:** `df.drop_duplicates()`

- **Example:** `df.drop_duplicates()`
- **groupBy()** (`pyspark.sql.DataFrame`)
 - **Functionality:** Groups data rows by designated keys to allow subsequent aggregate functions.
 - **Syntax:** `df.groupBy("col1", "col2")`
 - **Example:** `df.groupBy("category")`
- **agg()** (`pyspark.sql.DataFrame`)
 - **Functionality:** Computes multiple structural column aggregations across a grouped set of rows.
 - **Syntax:** `df.groupBy("key").agg(func1, func2)`
 - **Example:** `df.groupBy("sku").agg(sum("qty").alias("total"))`
- **pivot()** (`pyspark.sql.GroupedData`)
 - **Functionality:** Rotates distinctive row values from a column into distinct standalone matrix headers.
 - **Syntax:** `df.groupBy("row_key").pivot("pivot_col", [optional_distinct_values]).agg(agg_func)`
 - **Example:** `df.groupBy("user_id").pivot("movie_id").agg(avg("rating"))`
- **orderBy()** / **sort()** (`pyspark.sql.DataFrame`)
 - **Functionality:** Sorts DataFrame records based on one or multiple column rules in ascending/descending layouts.
 - **Syntax:** `df.orderBy(col("col_name").desc())`
 - **Example:** `df.orderBy(col("return_rate").desc(), col("user_id").asc())`
- **limit()** (`pyspark.sql.DataFrame`)
 - **Functionality:** Restricts the total row-count output of a DataFrame to a specified integer value.
 - **Syntax:** `df.limit(integer_value)`
 - **Example:** `top_five_df = df.orderBy(col("score").desc()).limit(5)`

4. Aggregative, Math, & String Functions

Computational blocks running inside aggregate wrappers or programmatic transformations.

- **sum()** (pyspark.sql.functions)
 - **Functionality:** Calculates the cumulative sum total of numeric elements across columns.
 - **Syntax:** `sum(col("col_name"))`
 - **Example:** `df.agg(sum(col("sales")).alias("total_sales"))`
- **count()** (pyspark.sql.functions)
 - **Functionality:** Counts the number of elements or records in a group.
 - **Syntax:** `count(col("col_name"))`
 - **Example:** `df.agg(count("*").alias("total_records"))`
- **countDistinct()** (pyspark.sql.functions)
 - **Functionality:** Returns the total count of completely unique, non-null values inside a group.
 - **Syntax:** `countDistinct("col_name")`
 - **Example:** `df.agg(countDistinct("order_id").alias("unique_orders"))`
- **avg()** (pyspark.sql.functions)
 - **Functionality:** Determines the mathematical average (mean) value of a designated numeric column.
 - **Syntax:** `avg(col("col_name"))`
 - **Example:** `df.agg(avg(col("price")))`
- **round()** (pyspark.sql.functions)
 - **Functionality:** Rounds floating-point numbers to a specified number of decimal places.
 - **Syntax:** `round(col("col_name"), scale_integer)`
 - **Example:** `df.withColumn("rounded_tax", round(col("tax"), 2))`
- **explode()** (pyspark.sql.functions)
 - **Functionality:** flattens a column containing nested arrays or maps into independent standalone rows for every element.
 - **Syntax:** `explode(col("array_column"))`

- **Example:** `df.withColumn("item", explode(col("items_array")))`
-

5. DateTime & Temporal Conversions

Handles parsing strings, building intervals, and altering time zones/formats.

- **to_date()** (`pyspark.sql.functions`)
 - **Functionality:** Converts a timestamp or formatted text string column into a standard `DateTime` (YYYY-MM-DD).
 - **Syntax:** `to_date(col("string_col"), [optional_format])`
 - **Example:** `df.withColumn("order_dt", to_date(col("order_date")))`
 - **date_format()** (`pyspark.sql.functions`)
 - **Functionality:** Extracts and reformats dates/timestamps into explicit regional custom text arrangements.
 - **Syntax:** `date_format(col("timestamp_col"), "format_pattern")`
 - **Example:** `df.withColumn("time_string", date_format(col("check_in"), "HH:mm:ss"))`
-

6. Window Analytic Blocks

Executes analytical tasks across multi-row sets partitioned by logical boundary groupings.

- **Window.partitionBy()** (`pyspark.sql.window.Window`)
 - **Functionality:** Configures a structural boundary layout for computing sliding/fixed window frames across rows.
 - **Syntax:** `Window.partitionBy("group_col").orderBy("sort_col")`
 - **Example:** `w = Window.partitionBy("symbol").orderBy("date")`
 - **lag()** (`pyspark.sql.functions`)
 - **Functionality:** Pulls values from an offset position located multiple steps prior to the current row index within a window.
 - **Syntax:** `lag(col("col_name"), offset_integer).over(window_specification)`
 - **Example:** `df.withColumn("prev_amt", lag("amount", 1).over(w))`
-

7. Relations, Tuning, & Performance Optimization

Involved in optimizing plans, indexing schemas, caching memory states, and handling structural keys.

- **join()** (pyspark.sql.DataFrame)
 - **Functionality:** Merges two separate DataFrames based on explicit conditional joining keys.
 - **Syntax:** `df1.join(df2, on=["keys"], how="join_type")`
 - **Example:** `df1.join(df2, on=["warehouse", "sku"], how="left")`
- **broadcast()** (pyspark.sql.functions)
 - **Functionality:** Flags a small DataFrame to be fully replicated into worker nodes, preventing shuffle overhead during joins.
 - **Syntax:** `join(broadcast(small_df), "key")`
 - **Example:** `df_large.join(broadcast(df_small), "url", "left")`
- **createOrReplaceTempView()** (pyspark.sql.DataFrame)
 - **Functionality:** Registers a DataFrame as a temporary local SQL view scoped to the active Spark session.
 - **Syntax:** `df.createOrReplaceTempView("view_name")`
 - **Example:** `df.createOrReplaceTempView("orders")`
- **cache()** (pyspark.sql.DataFrame)
 - **Functionality:** Persists DataFrame row schemas and partitions into fast in-memory execution states for repeated queries.
 - **Syntax:** `df.cache()`
 - **Example:** `df.cache()`
- **explain()** (pyspark.sql.DataFrame)
 - **Functionality:** Prints the execution plan (logical and physical) to the console for performance optimization.
 - **Syntax:** `df.explain("mode_string")`
 - **Example:** `df.explain("formatted")`

8. DataType Definitions (pyspark.sql.types)

Used to explicitly define structural schemas instead of relying on inferSchema.

- **StructType:** Defines a full layout schema blueprint. (StructType([StructField(...), ...]))
- **StructField:** Represents a single structural field element. (StructField("name", DataType(), isNullable))
- **StringType:** Standard textual data fields.
- **DoubleType:** 64-bit precision floating-point numeric columns.
- **TimestampType:** Combined year-month-day-hour-minute-second metrics.